American Terawatt

DEVELOPER INFRASTRUCTURE

Local LLM on the Shared GPU Pool

Claiming GPUs, staging the runtime and serving an inference endpoint

End-to-end account of how a local large language model is deployed on the shared GPU pool: how GPUs are borrowed, why the work happens inside a container, where files must live to survive, how the model runtime is installed into an image without package manager access, and how the resulting endpoint is reached from a remote machine.

Revision history

REV.	DATE	AUTHOR	DESCRIPTION
0.1	Sep. 17, 2026	Eugen Starschich	Initial document

CONTENT

1 Architecture 4
1.1 Scope 4
1.2 Hosts involved 4

2 Borrowing GPUs 5
2.1 The verbs 5
2.2 What a claim actually creates 5
2.3 Why a container at all 5
2.4 Consequences to plan around 6

3 Storage: what survives 7

4 Installing the runtime 8
4.1 Staging the binary 8
4.2 Where Ollama sits 8

5 Fetching and serving the model 9
5.1 Start the server 9
5.2 Pull the weights 9
5.3 Measured behaviour 9

6 Remote access 10

7 Teardown and restore 11

8 Pitfalls 12

9 Command reference 13

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

1 Architecture

1.1 Scope

This document records a working deployment of `gpt-oss:120b` under Ollama on one NVIDIA RTX PRO 6000 Blackwell card of the shared GPU pool. All concrete figures quoted in Section 5 come from that deployment.

1.2 Hosts involved

Four hosts participate. Nothing about the model runs on the developer VM; it is a control point and an SSH waypoint only.

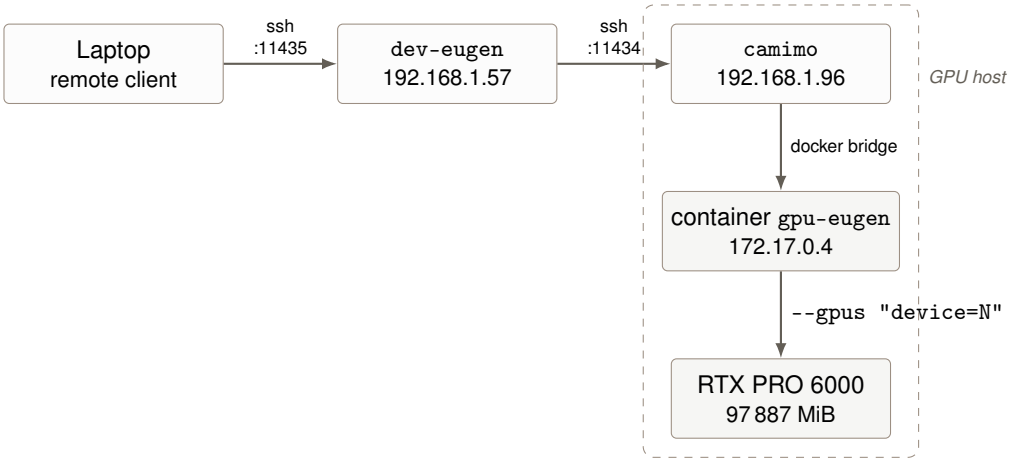


Figure 1: Hosts, container and card, with the two SSH hops that carry the endpoint out to the client.

Host	Role	Notes
Laptop	remote client	reaches the pool only via SSH
`dev-eugen`	developer VM	4 cores, 8 GB RAM, *no GPU*
`camimo`	GPU host	8 GPUs, also the NFS server for `/shared`
`gpu-eugen`	per-claim container	128 cores, 440 GB RAM visible

Table 1: The four hosts and what each contributes.

The GPU host and the storage host are the same machine: `/shared` is local disk there, and NFS-mounted on the developer VM.

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

2 Borrowing GPUs

2.1 The verbs

```
gpu status        # who holds what, what is free
gpu claim 2       # take 2 (all-or-nothing: 2 or none)
gpu shell         # shell inside your container
gpu shell CMD     # run one command in it, non-interactively
gpu release       # give them back
```

`gpu` on the developer VM is a thin relay: it re-quotes its arguments and forwards them over SSH to `camimo`, where the real tool runs. Because it requests a remote TTY only when it has one, it works from scripts and from automated callers as well as interactively.

2.2 What a claim actually creates

`gpu claim` allocates cards in pool state and then starts a container pinned to exactly those cards:

```
docker run -d --name gpu-eugen \
  --gpus "device=1" \
  --label gpu.owner=eugen --label gpu.devices=1 \
  --restart no \
  --user 3003:3000 \
  -e HOME=/home/eugen \
  -v /shared:/shared \
  --shm-size 16g \
  nvidia/cuda:12.6.0-base-ubuntu24.04 sleep infinity
```

Pool state lives in `/var/lib/gpu-pool/state.json` under a `flock`, and every command reconciles that file against `docker ps` before reporting or allocating. A container that dies therefore returns its cards on its own. If the state file is unreadable the tool refuses to write rather than falling back to “the pool is empty”, which would hand out cards already in use.

2.3 Why a container at all

Property	Consequence
`--gpus "device=N"`	Isolation is structural, not by convention. Other cards are invisible; yours renumbers to 0. A stray `CUDA_VISIBLE_DEVICES` cannot reach a colleague’s run.
Container is the unit of accounting	`gpu release` is `docker rm -f`; reconciliation against `docker ps` frees the cards.
No reaper, by design	A claim is released by its owner, never by a timer — so a long training run is safe, and a forgotten claim blocks the pool.
All-or-nothing allocation	Asking for 4 with 3 free yields nothing, rather than a half-sized allocation that silently invalidates the run.
`--user uid:3000`	Files created on `/shared` are owned by you in the `team` group, not by root.

Table 2: What the container arrangement buys.

2.4 Consequences to plan around

The image is `nvidia/cuda:12.6.0-base-ubuntu24.04`. The *base* variant carries the CUDA runtime and nothing else.

Constraint	Why
No `python3`, `curl`, `git`, `tmux`	`-base` image variant
No `sudo`; `apt install` fails	runs as your uid, not root
Service ports unreachable from outside	default docker bridge, isolated netns
Filesystem is ephemeral	`gpu release` deletes the container

Table 3: Constraints imposed by the claim container.

Anything that must run inside the container therefore has to be a self-contained binary staged in from outside.

3 Storage: what survives

Exactly one path spans both machines.

Path	Shared with container?	Survives `gpu release`?
`/shared/$USER`	yes, live and bidirectional	yes
`~` on the developer VM	**no**	n/a
`~` inside the container	it is *camimo's* home	yes, on camimo
container root filesystem	—	no

Table 4: Which paths are shared and which survive a release.

This deserves emphasis because it is easy to get wrong. The claim command bind-mounts `-v $HOME:$HOME`, but `$HOME` is resolved *on the GPU host*. The home directory visible inside the container is camimo's `/home/$USER`, not the developer VM's. A file written to `~` in one place is not visible in the other. Only `-v /shared:/shared` is unconditional, and only `/shared` genuinely spans both.

`/shared` **is not a backup.** One unmirrored NVMe copied hourly to another machine. Model weights belong there because they are large and re-downloadable; anything whose loss would matter belongs in Git, pushed elsewhere. It is also readable by the whole team, so it must never hold credentials, keys, tokens, draft commercial terms, or material under NDA.

4 Installing the runtime

4.1 Staging the binary

Because the container has no downloader and no root, the runtime is fetched on the developer VM and staged through `/shared`.

```
# on the developer VM
mkdir -p /shared/$USER/opt /shared/$USER/ollama-models
cd /shared/$USER/opt
curl -sL -o ollama.tar.zst \
  https://github.com/ollama/ollama/releases/download/v0.34.0/ollama-linux-amd64.tar.zst
mkdir -p ollama && zstd -dc ollama.tar.zst | tar -xf - -C ollama
```

This yields `bin/ollama` plus the bundled CUDA runners in `lib/ollama`. No installation step, no systemd unit, no root required — which is precisely why it works in a container where none of those are available.

4.2 Where Ollama sits

Ollama is a management layer, not the inference engine and not the model.

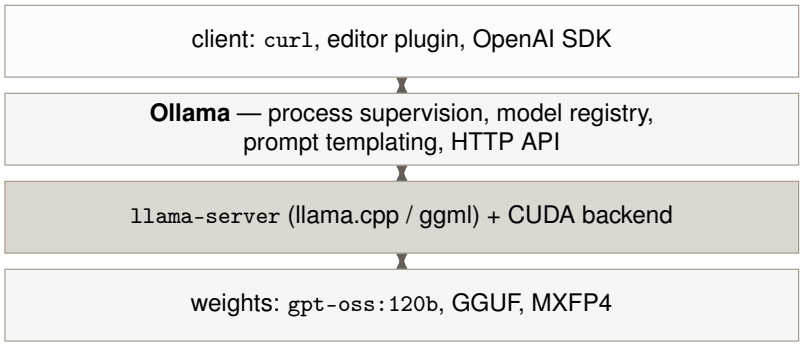


Figure 2: The serving stack, from client call down to the weights on disk.

Ollama supervises a separate `llama-server` process; llama.cpp and its CUDA kernels perform the arithmetic. Ollama contributes four things: it fetches and stores weights as blobs and manifests, it loads and unloads models with a keep-alive timer, it renders chat turns into the exact token format the model was trained on, and it exposes both a native API and an OpenAI-compatible one. It also sizes the context window to measured VRAM at start-up.

For a single user on a single card this is the right trade. For a shared, heavily concurrent endpoint, vLLM's continuous batching would serve better at the cost of a much heavier install.

5 Fetching and serving the model

5.1 Start the server

Keep this in a script on `/shared`. Long quoted one-liners do not survive the relay's re-quoting.

```
#!/bin/bash
# /shared/$USER/start-ollama.sh   --  run: gpu shell /shared/$USER/start-ollama.sh
set -u
BIN=/shared/$USER/opt/ollama/bin/ollama
LOG=/shared/$USER/ollama-serve.log

export OLLAMA_MODELS=/shared/$USER/ollama-models
export OLLAMA_HOST=0.0.0.0:11434

if ps -eo cmd | grep -q "[b]in/ollama serve"; then
    echo "already running"; exit 0
fi
cd /shared/$USER
nohup setsid "$BIN" serve > "$LOG" 2>&1 < /dev/null &
echo "launched pid $!"
```

5.2 Pull the weights

```
gpu shell bash -lc 'export OLLAMA_MODELS=/shared/$USER/ollama-models; \
  /shared/$USER/opt/ollama/bin/ollama pull gpt-oss:120b'
```

Roughly 65 GB. Because `OLLAMA_MODELS` points at `/shared`, the pull happens once and is reused by every later claim — and by teammates.

5.3 Measured behaviour

Quantity	Value
Model	`gpt-oss:120b`, 116.8 B parameters, MXFP4
Download	65 GB at ≈106–117 MB/s
Cold load from `/shared`	43 s
VRAM resident	67.0 GB of 97.9 GB
Generation, warm	183.5 tokens/s
Default context	131 072 tokens
Keep-alive	5 minutes idle, then unloaded

Table 5: Measured behaviour of `gpt-oss:120b` on one RTX PRO 6000.

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

6 Remote access

The server binds inside the container's network namespace, which nothing outside can reach. Two SSH forwards carry it out. Every hop terminates on loopback, so no inference port is ever published to the office network.

```
# 1. in the container: bind the bridge interface
gpu shell /shared/$USER/start-ollama.sh

# 2. on the developer VM: forward to the container
ssh -f -N -L 127.0.0.1:11434:172.17.0.4:11434 camimo

# 3. on the laptop: forward to the developer VM
ssh -N -L 11435:127.0.0.1:11434 eugen@192.168.1.57

# 4. verify, from a second terminal
curl http://localhost:11435/api/tags
```

The client then uses `http://localhost:11435/v1` as an OpenAI-compatible base URL with any non-empty API key, or `/api/...` for the native interface.

Security. Ollama performs no authentication whatsoever. Whoever reaches the port can run inference on the card and pull or delete models. Binding `0.0.0.0` inside the container exposes it to every account on the GPU host — acceptable within a team, but a deliberate decision rather than a default. Binding `0.0.0.0` on the developer VM would publish an unauthenticated endpoint to the whole LAN and should not be done. Access from outside the office requires a VPN; the developer VM has no public address.

7 Teardown and restore

```
gpu release        # deletes the container; cards return to the pool
```

The container, the running server and both tunnels are destroyed. The Ollama installation, the model weights and the start-up script all live on `/shared` and survive. Restoring service after a later claim costs two commands and no re-download:

```
gpu claim 1
gpu shell /shared/$USER/start-ollama.sh
ssh -f -N -L 127.0.0.1:11434:172.17.0.4:11434 camimo
```

Confirm the bridge address first, as it may differ between claims:

```
ssh camimo "docker inspect -f \
  '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' gpu-eugen"
```

8 Pitfalls

Symptom	Cause and remedy
Claim never released	Claims do not expire, by design. A forgotten claim blocks the pool indefinitely. Release explicitly.
`Text file busy` on a freshly staged binary	NFS write-back lag on a new executable under `/shared`. It clears by itself; wait rather than re-copying.
`pgrep -f "ollama serve"` always matches	The search pattern appears in the searching command's own command line. Use `grep "[b]in/ollama serve"`.
Quoted one-liner silently does nothing	The relay re-quotes arguments; nested quoting does not survive. Put the commands in a script on `/shared`.
`curl` returns the wrong model	A local Ollama already occupies port 11434, so the forward failed to bind and the query answered locally. Forward to a free local port such as 11435.
`ssh -N` appears to hang	Expected. `-N` runs no remote command; it holds the tunnel open and prints nothing. Test from a second terminal.
Work lost on release	Only `/shared` spans both machines. The container home is the GPU host's, not the developer VM's.
`<defunct>` processes accumulate	Container PID 1 is `sleep infinity` and never reaps children. Harmless; cleared on release.

Table 6: Symptoms seen in practice, with cause and remedy.

9 Command reference

```
# --- pool ------------------------------------------------------------
gpu status
gpu claim 1
gpu shell                       # interactive
gpu shell /path/to/script.sh    # one command
gpu release

# --- inside the container --------------------------------------------
nvidia-smi
ps -eo pid,etime,cmd | grep "[b]in/ollama serve"
tail -f /shared/$USER/ollama-serve.log

# --- model -----------------------------------------------------------
ollama list
ollama pull  gpt-oss:120b
ollama run   gpt-oss:120b --verbose "prompt"
ollama stop  gpt-oss:120b       # unload from VRAM, keep on disk

# --- endpoint --------------------------------------------------------
curl http://localhost:11435/api/tags
curl http://localhost:11435/api/generate \
  -d '{"model":"gpt-oss:120b","prompt":"...","stream":false}'
```